

Bridge System Language Roadmap

Platform Version: 0.0.4

Author: Meow Li

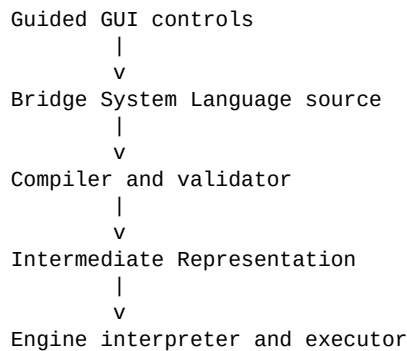
Copyright: Copyright (c) 2026 Meow Li. All Rights Reserved.

This document defines the roadmap for moving from direct YAML gadget files toward a layered authoring system with a Bridge System Language, a strict intermediate representation, and GUI tools.

1. Core Decision

Partner should separate human authoring from engine execution.

Target architecture:



The engine executes the strict intermediate representation. The Bridge System Language is an authoring layer, not the runtime object.

YAML remains useful as the current file format for the intermediate representation. In the future, YAML can continue as the readable serialization format for compiled rule objects, tests, import/export, and debugging.

2. Definitions

Bridge System Language

Bridge System Language, or BSL, is a formal domain-specific language for describing bridge bidding agreements.

BSL should feel closer to bridge writing than raw YAML, but it must not be free English. It needs grammar, keywords, validation, and a compiler.

Example style:

```
after 1N P:
  call 2D:
    transfer to H
    applies when responder.length(H) >= 5
    creates transfer target H status pending
    opener must accept_transfer_or_superaccept H
    alertable ACBL
```

BSL should use the ontology vocabulary from `docs/semantic\_ontology.md`.

Intermediate Representation

The intermediate representation, or IR, is the strict structured form that the engine can execute.

Current YAML gadget rules are an early version of this IR.

Example IR shape:

```
context:
  auction_pattern: "1NP"
call: 2D
applies_when:
  hand:
    H:
      min_length: 5
meaning:
  call_nature: artificial
  action_type: transfer
  target_suit: H
effects:
  - create_transfer:
      target_suit: H
      status: pending
```

The IR should be explicit, normalized, validated, and executable. It is acceptable if technical users prefer editing IR directly.

Compiler

The compiler converts BSL into IR.

Compiler responsibilities:

- ? parse BSL syntax,
- ? resolve bridge keywords against the ontology,
- ? normalize calls and auction patterns,
- ? produce IR,
- ? validate required fields,
- ? detect ambiguous or unsupported phrases,
- ? preserve source locations for error messages,
- ? emit warnings for incomplete agreements.

Interpreter

The interpreter is the engine layer that executes IR.

Interpreter responsibilities:

- ? replay auction history,
- ? build semantic state,

- ? match rule context,
- ? evaluate `requires` and `applies\_when`,
- ? compare candidate calls,
- ? update semantic state through effects,
- ? produce public meaning, internal origin, and diagnostics.

3. Important Product Rule

BSL is not always easier than IR.

Different users and tasks need different surfaces:

- ? Casual users may prefer guided forms.
- ? Bridge writers may prefer BSL.
- ? Technical users may prefer IR/YAML.
- ? AI agents may generate BSL first, then compile and validate.
- ? Test authors may prefer direct IR/YAML fixtures.

Therefore the GUI should support multiple synchronized views instead of forcing one authoring mode.

Target authoring model:

Form View <-> BSL View <-> IR/YAML View

The system must make clear which view is authoritative for a given edit. The safest default is:

1. form edits generate or update BSL,
2. BSL compiles to IR,
3. IR is executed by the engine,
4. direct IR edits are allowed only in advanced mode and must be validated.

4. GUI Responsibilities

The GUI should be a bridge-system editor, not a raw file editor.

Form View

Audience:

- ? casual users,
- ? teachers,
- ? system authors who are entering common structures.

Expected behavior:

- ? provide bridge-aware controls for auction context, call, meaning, hand requirements, obligations, and alertability,
- ? generate BSL or IR from user selections,
- ? prevent impossible combinations when possible,
- ? explain validation errors in bridge language,
- ? show generated source for review.

Example form fields:

```
Auction context: after 1N P
Call: 2D
Meaning type: transfer
Target suit: H
Responder requirement: length(H) >= 5
Opener obligation: accept or superaccept
Alert status: ACBL alertable
```

BSL View

Audience:

- ? advanced bridge users,
- ? system designers,
- ? users editing imported notes,
- ? LLM-assisted workflows.

Expected behavior:

- ? provide syntax highlighting,
- ? autocomplete ontology keywords,
- ? autocomplete calls and suits,
- ? show compile errors with source locations,
- ? show the compiled IR preview,
- ? support comments and readable organization.

IR/YAML View

Audience:

- ? engineers,
- ? AI agents,
- ? power users,
- ? test authors.

Expected behavior:

- ? show normalized compiled rules,
- ? allow direct editing in advanced mode,

- ? validate against the IR schema,
- ? show rule origins and semantic effects,
- ? support diffing and debugging.

The IR/YAML view should not be hidden, because some complex rules are clearer when written structurally.

5. Source Of Truth Policy

Each gadget should eventually declare its source mode.

Recommended source modes:

- ? ``bsl_source``: BSL is authoritative; IR is compiled output.
- ? ``ir_source``: IR/YAML is authoritative; BSL may be generated as a readable view when possible.
- ? ``generated_draft``: created by LLM or import tooling and not yet approved.

The engine should only load approved, validated IR.

BSL source and IR output should both preserve author metadata, platform version, gadget version, rule origin, and source locations when possible.

6. Relationship To Ontology

The ontology is the type system and vocabulary for BSL and IR.

BSL keywords should map to ontology concepts:

- ? ``transfer`` maps to formal transfer state.
- ? ``agreed suit`` maps to ``agreed_suit``.
- ? ``notrump focus`` maps to ``notrump_focus``.
- ? ``game forcing`` maps to ``forcing_status: game_forcing``.
- ? ``keycard ask`` maps to ``keycard_context``.
- ? ``interference`` maps to competitive state.
- ? ``must`` creates an obligation.

If a BSL phrase cannot map to ontology or validated custom extension terms, the compiler should reject it or mark the rule as an unapproved draft.

7. Roadmap Milestones

Milestone 0: Current YAML Prototype

Status: current checkpoint.

The platform has:

- ? compact call and auction notation,
- ? compact hand strings,
- ? directory-based gadgets,
- ? YAML rule files,
- ? a starter 2/1 gadget,
- ? a starter four-way Jacoby transfer gadget,
- ? fixture-driven tests,
- ? public meaning and internal origin output.

YAML is currently both the authoring format and the executable structure.

Milestone 1: Formal IR Schema

Goal: define the strict rule object the engine executes.

Work:

- ? decide final IR fields such as `context`, `call`, `applies\_when`, `requires`, `selection`, `meaning`, `shows`, `effects`, and `clears`,
- ? define schema validation,
- ? normalize current YAML into the IR schema,
- ? document compatibility with current `selection`, `meaning`, and `semantic\_effects`.

Exit criteria:

- ? invalid rule files produce clear validation errors,
- ? every loaded rule has a normalized IR form,
- ? tests can inspect normalized IR.

Milestone 2: Ontology-Backed Semantic State

Goal: replace flexible ad-hoc semantic facts with formal semantic state.

Work:

- ? implement `SemanticState`,
- ? implement ontology effect handlers,
- ? implement ontology query handlers,
- ? support formal transfer state,
- ? support agreed-suit/trump and notrump-focus state,
- ? support forcing and obligation state,
- ? support competitive interference state,
- ? support ambiguity diagnostics.

Exit criteria:

- ? Jacoby transfer rules use formal transfer state,
- ? `4N` examples can be represented as semantic-resolution candidates,
- ? diagnostics identify ambiguous or missing semantic state.

Milestone 3: BSL Grammar Draft

Goal: design a small but real language syntax.

Work:

- ? define grammar for `after`, `call`, `applies when`, `requires`, `shows`, `creates`, `sets`, `clears`, `must`, and `alertable`,
- ? choose parser technology,
- ? implement source-location-aware parser errors,
- ? compile BSL into normalized IR,
- ? add BSL examples equivalent to current YAML examples.

Exit criteria:

- ? current 1N opening and heart-transfer example can be written in BSL,
- ? BSL compiles to the same normalized IR as hand-written YAML,
- ? malformed BSL gives useful errors.

Milestone 4: Round-Trip And Diff Tooling

Goal: make source transformations inspectable.

Work:

- ? display BSL source beside compiled IR,
- ? show semantic diff between rule versions,
- ? preserve comments where practical,
- ? preserve source locations from BSL to IR diagnostics,
- ? define when IR can be converted back to readable BSL.

Exit criteria:

- ? users can understand what a GUI or LLM-generated change did,
- ? tests verify BSL-to-IR compilation output.

Milestone 5: GUI Form Builder

Goal: support non-programmer authoring for common rules.

Work:

- ? create forms for auction context, call, meaning, hand requirements, obligations, and alertability,
- ? generate BSL or IR from form input,
- ? compile and validate before saving,
- ? show generated source and diagnostics,
- ? support common gadget templates.

Exit criteria:

- ? a user can create a simple transfer rule without writing YAML,
- ? validation errors are understandable to a bridge player.

Milestone 6: BSL Editor

Goal: support advanced text authoring.

Work:

- ? syntax highlighting,
- ? ontology keyword autocomplete,
- ? call and suit autocomplete,
- ? inline errors,
- ? compiled IR preview,
- ? test-run button for example hands and auctions.

Exit criteria:

- ? advanced users can write and validate BSL directly.

Milestone 7: IR/YAML Inspector And Advanced Editor

Goal: keep precise technical editing available.

Work:

- ? show normalized IR,
- ? allow advanced direct IR edits,
- ? validate schema on every save,
- ? show origin, semantic effects, and compiled output,
- ? support exporting IR/YAML.

Exit criteria:

- ? engineers and AI agents can debug the exact executable rule object.

Milestone 8: LLM-Assisted Drafting

Goal: convert natural bridge descriptions into draft BSL or IR.

Work:

- ? use LLM to draft BSL from natural-language system notes,
- ? compile generated BSL to IR,
- ? validate against ontology and schema,
- ? require user review and approval,
- ? show differences from existing gadget rules.

Exit criteria:

- ? LLM output is never silently activated,
- ? generated gadgets remain drafts until validated and approved.

Milestone 9: System Notes Import And Export

Goal: connect formal rules with human-readable system notes.

Work:

- ? generate readable system notes from IR,
- ? import or draft BSL from existing notes,
- ? support examples, warnings, and undefined continuations,
- ? preserve public disclosure separately from internal training details.

Exit criteria:

- ? users can move between system notes, BSL, IR, and engine behavior with clear validation.

8. Testing Requirements

Each language layer needs tests.

Required test categories:

- ? BSL parser accepts valid examples,
- ? BSL parser rejects malformed examples,
- ? BSL compiler emits expected IR,
- ? IR schema rejects invalid rule objects,
- ? ontology effects produce expected semantic state,
- ? GUI-generated rules compile to expected IR,
- ? LLM-generated drafts must remain unapproved by default,
- ? BSL and IR examples produce the same bidding behavior.

Human-readable test documentation must be updated whenever test fixtures change.

9. Open Design Questions

Questions to resolve before implementation:

1. What parser technology should BSL use?
2. Should BSL be indentation-based, block-based, or line-oriented?
3. How much free-form display text should BSL allow?
4. How should comments survive BSL-to-IR compilation?
5. Can every IR rule be represented in BSL, or should some advanced rules remain IR-only?
6. How should source mode be stored per gadget?
7. Should GUI forms generate BSL first or direct IR first for simple rules?

The current recommendation is to make BSL the preferred human-readable source, but to keep direct IR editing as a first-class advanced workflow.